

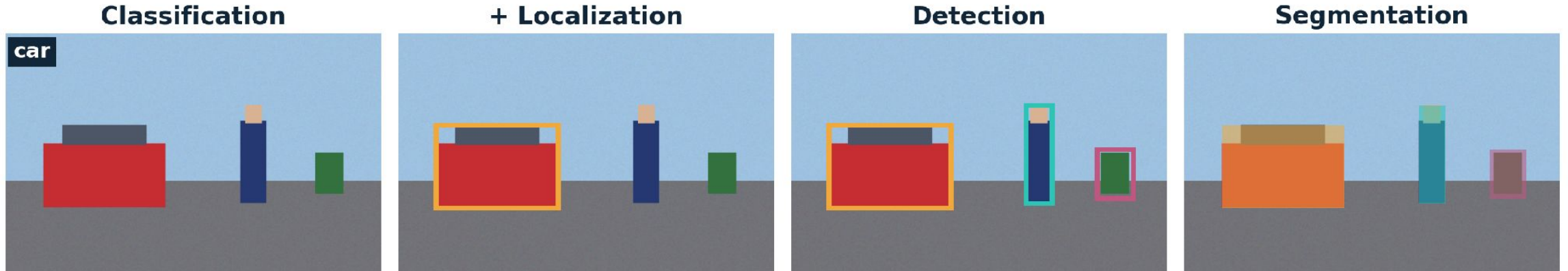
MODULE 4 · SESSION 1

Object Detection: Four Building Blocks

Boxes · Drawing · IoU · NMS — the code every detector runs on

AAIS-322 · Designing an Object Detection System

From “what is it” to “what, where, how many”



Your Module 1 classifier returns one label per image — always.

A detector returns a list of (box, class, score) triples — and the length of that list changes with every image. That single difference creates everything on the next slides.

A box is four numbers — but which four?

xyxy

`x_min, y_min, x_max, y_max`
Pascal VOC, torchvision

xywh

`x_min, y_min, width, height`
COCO annotations

YOLO

`cx, cy, w, h` — all 0-1
YOLO .txt labels (Session 3)

```
def xyxy_to_yolo(b, iw, ih):  
    x1, y1, x2, y2 = b  
    return (((x1+x2)/2)/iw,      # cx  
            ((y1+y2)/2)/ih,      # cy  
            (x2-x1)/iw,          # w  
            (y2-y1)/ih)          # h
```

Two traps that never crash — they just put boxes in the wrong place:

- `img.shape` is (height, width) but boxes are (x, y) — row/column and x/y are swapped
- Origin (0,0) is the TOP-left; y increases downward

One detection = box + class + score

```
def draw_boxes(img, boxes, labels=None,
               scores=None):
    ax = plt.gca(); ax.imshow(img)
    for i, (x1, y1, x2, y2) in enumerate(boxes):
        ax.add_patch(Rectangle((x1, y1),
                               x2-x1, y2-y1, fill=False,
                               edgecolor='orange', lw=2))
        ax.text(x1, y1-4,
                f"{labels[i]} {scores[i]:.2f}")
```

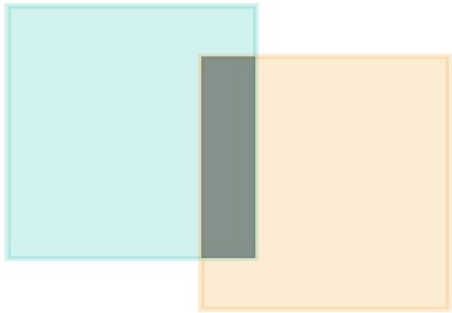
Why this helper matters

- You cannot debug a detector from numbers alone
- Used every single week from here on
- Shows the full triple: where, what, how sure

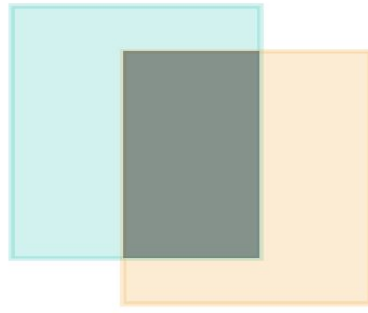
Rule of the module: after every model call, draw the boxes. Trust your eyes before you trust a metric.

IoU — how a box gets graded

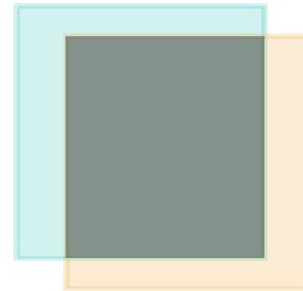
IoU = 0.10



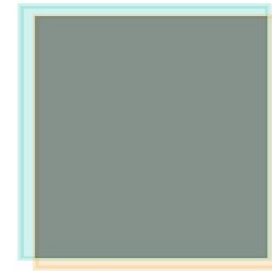
IoU = 0.29



IoU = 0.54



IoU = 0.82



```
def iou(a, b):  
    ix1, iy1 = max(a[0],b[0]), max(a[1],b[1])  
    ix2, iy2 = min(a[2],b[2]), min(a[3],b[3])  
    inter = max(0,ix2-ix1) * max(0,iy2-iy1)  
    union = area(a) + area(b) - inter  
    return inter/union if union else 0.0
```

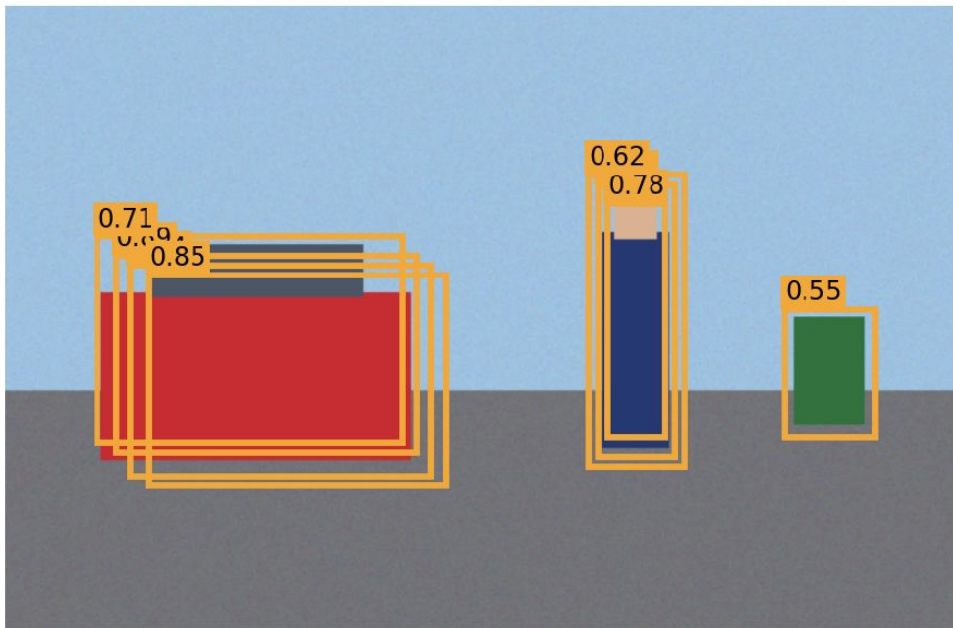
0.5 is a convention, not a law

Inherited from Pascal VOC. A parking counter can live with 0.5; a surgical robot cannot. IoU is also the duplicate test inside NMS and the threshold behind mAP.

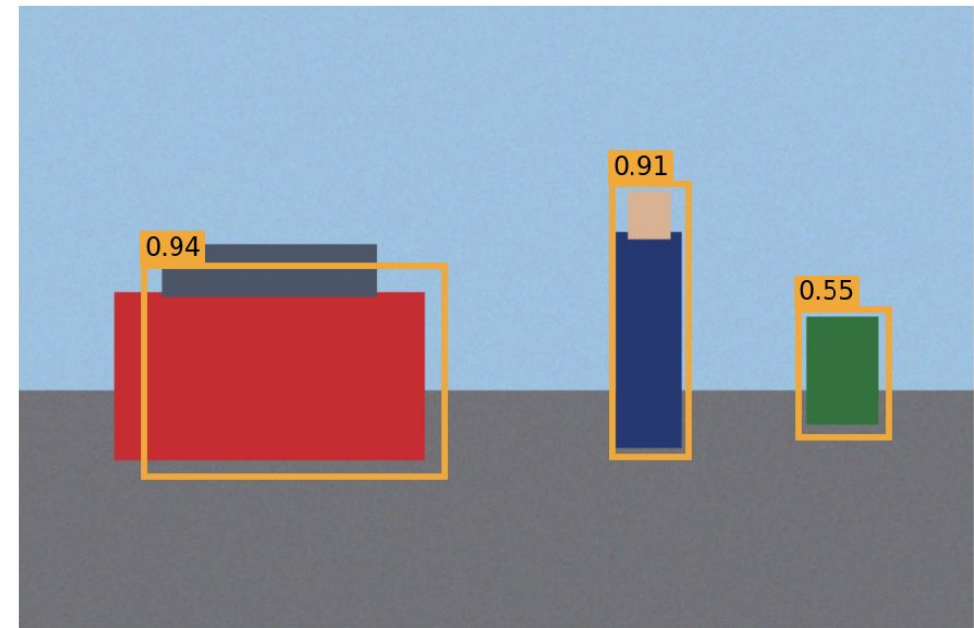
Detectors don't return one box per object

Every region near the car also looks like a car. Non-maximum suppression is the cleanup step.

Raw model output — 8 boxes



After NMS — 3 boxes



Runs inside every detector you will ever use — including YOLO in Session 3.

NMS is nine lines of Python

```
def nms(boxes, scores, thr=0.5):  
    order = sorted(range(len(boxes)),  
                    key=lambda i: scores[i], reverse=True)  
    keep = []  
    while order:  
        best = order.pop(0)  
        keep.append(best)  
        order = [i for i in order  
                  if iou(boxes[best], boxes[i]) < thr]  
    return keep
```

1 Sort by confidence

2 Keep the top box

3 Delete overlapping twins

4 Repeat until empty

The two knobs you will turn all module:

Confidence threshold (how sure before we report) and NMS IoU threshold (how much overlap means “same object”). A self-driving car and a ticket-issuing camera run the same model with opposite settings.

Saved once, used all module

`xyxy_to_yolo() + friends`

Session 3: converting labels for YOLO training

`draw_boxes()`

Every session: seeing what the model did

`iou()`

Sessions 2–3: grading predictions, computing mAP

`nms()`

Session 3: cleaning raw YOLO output

```
from detutils import iou, nms, draw_boxes    – no retyping, ever.
```

Four things to take with you

Boxes

three formats — always test the round trip

Drawing

box + class + score; trust your eyes first

IoU

$\text{overlap} \div \text{union}$ — the ruler behind everything

NMS

keep the best, delete its twins, repeat

Next session — object detection in autonomous systems: where these thresholds become safety decisions.